P2

Anatomía ELF/PE y direcciones

Parte I · RA2 · 2 h

Universidad de Almería · Edición de 27 de septiembre de 2026

Documento: Guion de práctica para el alumnado
Asignatura: Ingeniería Inversa y Análisis Forense Digital
Uso: Laboratorio docente en entorno controlado y sobre las muestras indicadas
Alumno/a: _____ Grupo: _____

Índice

1. Mapa de trabajo **2**

2. Material y preparación **2**

3. Trabajo en sesión (120 minutos) **2**

4. Entregable **2**

5. Evaluación **2**

6. Teoría necesaria para realizar la práctica **3**
6.1. Fichero, imagen cargada y direcciones 3

7. Software requerido **3**

8. Guía detallada de realización **3**
8.1. 1. Identificar los ficheros 3
8.2. 2. Inventariar ELF 4
8.3. 3. Comprobar una conversión ELF 4
8.4. 4. Inventariar y convertir PE 4
8.5. 5. Contrastar con la interfaz gráfica 4

9. Plantillas de cálculo y entrega **4**

10. Controles de calidad **4**

11. Actualización PE-01 disponible — 28 de septiembre de 2026 **5**
11.1. Material definitivo 5
11.2. Ejercicio con datos de esta edición 5

Estado actual: PE-01 1.0 disponible. La actualización del 28 de septiembre de 2026, al final del documento, contiene el PE manifestado y sus datos comprobados. Sustituye las referencias anteriores a su preparación pendiente; se conserva la ficha histórica.

1 Mapa de trabajo



Figura 1: Secuencia de trabajo y puntos de control de la práctica.

El diagrama resume el orden recomendado. Cada transición debe quedar respaldada por una anotación, salida o fichero verificable en la entrega.

2 h · RA2 · 5 % de la nota de Parte I. Guion completo en LaTeX. Rúbrica única en criterios de evaluación.

2 Material y preparación

ELF propio `ejemplos-x86-strip-opt` y PE benigno construido con el mismo fuente:

```
make -C muestras/build x86 pe
readelf -h -l -S muestras/build/salida/ejemplos-x86-strip-opt
readelf -d -r --dyn-syms muestras/build/salida/ejemplos-x86-strip-opt
objdump -p muestras/build/salida/ejemplos-pe.exe
```

`make pe` requiere MinGW-w64 en la VM del docente; el PE se puede entregar ya compilado con manifiesto. Binary Ninja lo analiza estáticamente sin Windows ni Wine.

3 Trabajo en sesión (120 minutos)

1. Hash, tamaño, arquitectura, clase y endianness (15 min).
2. ELF: entry point, `PT_LOAD`, `.text`, `.bss`, símbolos e imports (35 min).
3. Convertir una VA de enlace a offset usando el segmento correspondiente; comprobar los bytes con `xxd`. Explicar una dirección de `.bss` sin bytes en fichero (25 min).
4. PE: máquina, `ImageBase`, RVA del entry point e imports; calcular VA preferida y un offset válido desde una sección. Diferenciar base preferida y real (30 min).
5. Revisar por qué un import o timestamp no prueba actividad o fecha real (15 min).

4 Entregable

Ficha comparativa con hash y localizadores, dos conversiones comprobadas y un caso sin correspondencia en disco. Identificar bibliotecas y separar símbolos importados (no definidos) de símbolos definidos. No ejecutar muestras para obtener esta ficha.

5 Evaluación

Se aplica la rúbrica común: corrección técnica 40 %, evidencia reproducible 25 %, interpretación y límites 25 %, claridad 10 %. Todos los criterios se aplican al objetivo concreto de esta práctica; las ampliaciones no son necesarias para la máxima nota.

6 Teoría necesaria para realizar la práctica

6.1 Fichero, imagen cargada y direcciones

Un ejecutable contiene bytes y metadatos para que el cargador construya una imagen en memoria. Una sección organiza información para enlazado y análisis; un segmento describe rangos que el cargador mapea. No hay correspondencia obligatoria uno a uno. Una región puede ocupar memoria sin tener todos esos bytes en el fichero, como `.bss` o la parte para la que `p_memsz >p_filesz`.

Para una VA respaldada por bytes de un segmento ELF `PT_LOAD`:

```
offset = p_offset + (VA - p_vaddr)
```

Se valida primero que la VA esté dentro del segmento y del rango `p_filesz`. En un PIE, la dirección observada al ejecutar incorpora el *load bias*.

En PE, un RVA es relativo al `ImageBase` preferido:

```
VA_preferida = ImageBase + RVA
offset = PointerToRawData + (RVA - VirtualAddress_de_seccion)
```

La segunda fórmula exige elegir la sección que contiene el RVA y comprobar sus tamaños virtual y raw. ASLR puede cambiar la base real sin cambiar el RVA.

El entry point suele entrar en código de runtime antes de `main`. Un binario `stripped` puede conservar tabla dinámica, imports y cadenas. Un import acredita una dependencia prevista, no ejecución histórica. Los timestamps de cabecera son campos modificables y no fechas autenticadas.

7 Software requerido

Obligatorio: `file`, `sha256sum`, `readelf`, `objdump`, `xxd` y Binary Ninja Desktop o Ghidra. GCC/Make construyen el ELF. MinGW-w64 solo se necesita en el puesto que genere el PE; el alumnado puede recibirlo con manifiesto.

```
bash PreparaciónEntorno/entorno/provision.sh --check
readelf --version | head -n 1
objdump --version | head -n 1
```

No se ejecuta el PE y no se necesita Wine.

8 Guía detallada de realización

8.1 1. Identificar los ficheros

```
make -C muestras/build x86
mkdir -p resultados/P2
sha256sum muestras/build/salida/ejemplos-x86-strip-opt \
  > resultados/P2/elf.sha256
file muestras/build/salida/ejemplos-x86-strip-opt \
  > resultados/P2/file-elf.txt
```

Si MinGW-w64 está validado:

```
make -C muestras/build pe
sha256sum muestras/build/salida/ejemplos-pe.exe > resultados/P2/pe.sha256
file muestras/build/salida/ejemplos-pe.exe > resultados/P2/file-pe.txt
```

En otro caso use exactamente el PE entregado. Registre nombre, tamaño, hash, procedencia y herramienta de generación.

8.2 2. Inventariar ELF

```
readelf -h -l -S muestras/build/salida/ejemplos-x86-strip-opt \
  > resultados/P2/elf-estructura.txt
readelf -d -r --dyn-syms muestras/build/salida/ejemplos-x86-strip-opt \
  > resultados/P2/elf-dinamico.txt
```

Extraiga clase, endianness, máquina, tipo, entry point, `PT_LOAD`, secciones y bibliotecas. Cite la entrada de salida que sostiene cada valor. Distinga símbolos definidos e indefinidos.

8.3 3. Comprobar una conversión ELF

Elija una VA dentro de los bytes de un `PT_LOAD`. Copie `p_offset`, `p_vaddr`, `p_filesz` y `p_memsz`, valide el rango, calcule el offset y examine bytes:

```
# Sustituir OFFSET_CALCULADO por el hexadecimal obtenido en este build.
xxd -g 1 -s OFFSET_CALCULADO -l 32 \
  muestras/build/salida/ejemplos-x86-strip-opt
```

Contraste con `objdump -d`. Después seleccione una dirección de `.bss` y explique por qué puede existir en memoria sin bytes equivalentes almacenados.

8.4 4. Inventariar y convertir PE

```
objdump -p muestras/build/salida/ejemplos-pe.exe > resultados/P2/pe-cabeceras.txt
objdump -h muestras/build/salida/ejemplos-pe.exe > resultados/P2/pe-secciones.txt
```

Registre máquina, `ImageBase`, `AddressOfEntryPoint`, secciones e imports. Calcule la VA preferida. Identifique la sección que contiene el RVA, calcule el offset y compruebe bytes con `xxd`. No use `ImageBase + RVA` como offset.

8.5 5. Contrastar con la interfaz gráfica

Abra ambos ficheros y confirme arquitectura, entry point, imports y una cadena. La herramienta puede asignar nombres y pseudocódigo: marque esos datos como derivados. Compruebe al menos una dirección mediante cabeceras y bytes crudos.

9 Plantillas de cálculo y entrega

Formato	Dato	Valores usados	Fórmula	Resultado	Comprobación
ELF	VA elegida	completar	`p_offset+(VA-p_vaddr)`	completar	`xxd`/objdump
PE	RVA elegido	completar	`RawPtr+(RVA-SecVA)`	completar	`xxd`/objdump

Incluya identificación, tablas de estructura, dos conversiones, caso sin respaldo en disco, imports, límites y anexo de comandos.

10 Controles de calidad

- Verificar el rango antes de aplicar la fórmula.
- Diferenciar tamaño virtual y raw.
- Tratar `AddressOfEntryPoint` como RVA, no como VA absoluta.
- No afirmar que `stripped` elimina todas las cadenas o imports.

- No inferir ejecución a partir de una dependencia.
- No copiar direcciones o hashes de otra compilación.

11 Actualización PE-01 disponible — 28 de septiembre de 2026

El PE benigno ya está producido y manifestado. Esta sección concreta el material de la práctica y sustituye las referencias anteriores a su preparación pendiente. Se conserva íntegro el guion previo. El código C del PE es exactamente el mismo ejemplos.c empleado para construir el ELF; los builds y sus direcciones son distintos.

11.1 Material definitivo

Descargue el paquete del alumnado PE-01. Contiene ejemplos-pe.exe, MANIFIESTO.json, SHA256SUMS, instrucciones y avisos del runtime. El ejecutable tiene 39.936 bytes. No necesita compilarlo ni instalar Windows o Wine. El manifiesto de distribución no es un recurso Windows RT_MANIFEST ni una firma Authenticode. Fuente y símbolos quedan en el paquete docente.

Desde una carpeta nueva, extraiga el paquete y analice la copia distribuida:

```
tar -xzf /RUTA/PE-01-alumnado.tar.gz
cd alumnado
sha256sum -c SHA256SUMS
mkdir resultados
file ejemplos-pe.exe > resultados/formato.txt
objdump -p ejemplos-pe.exe > resultados/cabeceras.txt
objdump -h ejemplos-pe.exe > resultados/secciones.txt
```

Sustituya /RUTA por la ubicación recibida. Estos comandos leen el PE sin ejecutarlo. Conserve MANIFIESTO.json y los logs junto a su ficha. La fuente de referencia de los campos es la especificación PE de Microsoft.

11.2 Ejercicio con datos de esta edición

1. Identifique Machine, Magic y Subsystem. Explique la diferencia entre PE32 y PE32+; no deduzca la arquitectura únicamente de la extensión .exe.
2. Registre ImageBase, AddressOfEntryPoint, SectionAlignment y FileAlignment. Un RVA es relativo al comienzo de la imagen; la VA efectiva sumaría la base real de carga, mientras que aquí solo puede calcularse la VA preferida.
3. Localice la sección del entry point. Para leer 16 bytes valide delta >= 0, delta+16 <= VirtualSize y delta+16 <= SizeOfRawData, y calcule RawPtr+delta.
4. Compruebe esos bytes con xxd y con el desensamblador. Si este usa otra base de análisis, registre el cambio antes de comparar sus direcciones.
5. Elija el primer RVA de .bss. Explique si tiene bytes de fichero; no trate un PointerToRawData nulo como permiso para leer la cabecera como datos de .bss.
6. Identifique DLL y funciones importadas, e indicadores de ASLR/DEP. Separe código de aplicación y runtime, símbolos definidos e imports. Un import no prueba uso.
7. Examine el timestamp COFF. Está fijado a cero por el procedimiento de generación: explique por qué no acredita una compilación o ejecución histórica en 1970.

```
RVA del entry point ---- + ImageBase ----> VA preferida
          |
          +-- buscar seccion --> delta = RVA - VirtualAddress
```

```
                    |
            validar rango raw
                    |
            offset = RawPtr + delta
```

Complete la comparación con el ELF del guion y entregue los dos cálculos comprobados, el caso sin bytes y las limitaciones. El análisis estático no observa una base real de Windows ni acredita una ejecución histórica de este PE.